

FIELD REPORT · AUGUST 2026

Making a 27B Agentic Model Practical on an Intel Arc Pro B70

Repeated profiling, configuration tuning, and real coding-agent evaluations across Pi, Pi-Wiggum, and OpenCode.

Bottom line: the B70 became a credible local agentic coding machine. The winning Qwen3.8 profile combines f16 KV cache, one-token MTP speculation, a long reasoning ceiling, and zero presence penalty. It recovered deep-context speed and passed every behavioral check we applied.

33.8 t/s

f16 KV + MTP at ~16K depth, 86% draft acceptance

20.3% faster

Pi-Wiggum wall time after tuning

98 / 100

human score for Pi-Wiggum from the operator

Hardware: Intel Arc Pro B70 (32 GB) in a USB4 eGPU enclosure · llama.cpp SYCL/Level Zero/XMX · llama-swap · OpenAI-compatible local endpoint

Executive summary

The original goal was to make a large local model useful for agentic coding on a Battlemage B70 rather than merely impressive in a short benchmark. The work proceeded as a loop: update llama.cpp, rebuild against Intel oneAPI, measure context-depth behavior, test speculative decoding, test sampling choices, verify tool and vision paths, then run the same coding task through multiple agents.

The central finding was that the slow path was not the model's raw compute. It was the SYCL quantized KV-cache flash-attention path as context grew. On Qwen3.8, q8 KV decode dropped from 26.6 t/s at an empty context to 15.5 t/s around 16K tokens. Changing only the KV cache to f16 raised that deep-context result to 22.2 t/s; adding one-token MTP reached 33.8 t/s with 86% acceptance.

The final recommendation is deliberately profile-based: use `qwen3.8-27b-think` for 128K reasoning-heavy agent work, `qwen3.8-27b-mtp` when 256K context matters more, and `qwen3.6-35b-a3b UD-Q4_K_S` as the balanced general-purpose Qwen option.

Hardware and software baseline

Component	Observed setup	Why it mattered
GPU	Intel Arc Pro B70, Battlemage, 32 GB GDDR6; ~31,023 MiB exposed	Large enough for 27–35B Q4 models and meaningful KV-cache experiments.
Transport	USB4 eGPU enclosure	Cold model loads are tens of seconds; warm inference is primarily a GPU bandwidth problem.
Backend	llama.cpp SYCL → Level Zero → XMX, oneDNN flash attention, host-memory fallback	Uses Intel hardware acceleration instead of the much slower Vulkan path.
Refresh	llama.cpp 7e4c0a9 → 2e92ecd02 , binary build 1579	Included quantized SYCL copy-kernel corrections and Hadamard-hint support, along with the preceding Gated DeltaNet/MTP work.
Build policy	IntelLLVM 2026.1.1; GGML_SYCL_DEVICE_ARCH intentionally unset	JIT avoids the large-buffer limitation encountered with the attempted AOT BMG target; the compiled kernels cache after first use.

What repeated profiling taught us

Experiment	Observation	Decision
KV precision at depth	q8: 26.6 t/s empty → 15.5 t/s at ~16K; f16: 22.2 t/s at depth (+43%).	Spend the cache budget on f16 KV for the 128K thinking profile.
MTP draft length	One draft token beat two and three in matched runs; one-token f16 MTP reached 33.8 t/s and 86% acceptance.	Keep <code>--spec-type draft-mtp --spec-draft-n-max 1</code> .
n-gram drafter	Combined MTP + ngram-mod fell to 31.4 t/s with 68% acceptance.	Do not enable it on this SYCL stack.
Presence penalty A/B	0 completed the seeded code task in 2,733 tokens; 0.5 omitted an edge-case test; 1.5 exhausted the cap and returned defective code.	Use <code>--presence-penalty 0</code> for Qwen3.8.
Reasoning control	<code>reasoning_effort=low</code> was the verified short-reasoning setting. <code>none</code> consumed its allowance in hidden reasoning and returned no final answer.	Use low/medium for latency-sensitive work; keep the server ceiling as a safety bound.

Prompt processing	About 844 t/s at 512 tokens, but tiny uncached prompts can be ~67 t/s because of per-call/USB4 overhead.	Keep <code>--parallel 1</code> and let warm prompt caching help growing agent conversations.
-------------------	--	--

Model configuration that won

Alias	Model / quant	Context and cache	Role
qwen3.8-27b-think	Unsloth Qwen3.8-27B Q4_K_S with F16 projector and MTP tensors	131,072; f16 KV; MTP n=1; reasoning budget 8,192	Daily agentic driver for Pi/OpenCode and long reasoning.
qwen3.8-27b-mtp	Same GGUF	262,144; q8 KV; MTP n=1; reasoning budget 2,048	256K-context option; less attractive at deep context.
qwen3.8-27b	Same GGUF	262,144; q8 KV; no speculation	Conservative non-speculative fallback.
qwen3.6-35b-a3b	Unsloth UD-Q4_K_S MoE + F32 projector	262,144; q8 KV	Balanced general-purpose Qwen; ~55 t/s short tool-call test.

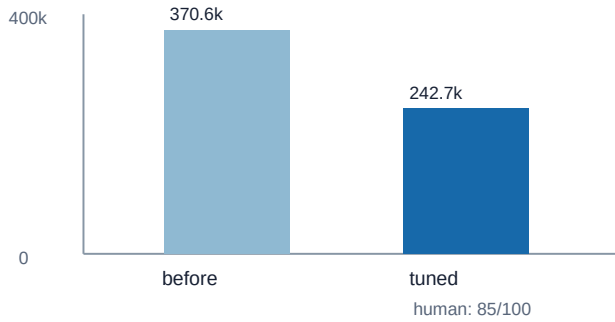
Important memory trade-off: the Qwen3.8 hybrid model has only 16 full-attention layers, but f16 KV still costs about 64 KiB per token. That is roughly 8 GiB at 128K and 16 GiB at 256K. The B70 can afford 128K f16 KV beside the weights, but 256K f16 KV is not practical.

Agentic client comparison

These are end-to-end task runs using the same office simulation prompt. The baseline used the older `qwen3.8-27b` profile; the tuned run used `qwen3.8-27b-think`. The comparison therefore measures the practical profile change, not one isolated flag.

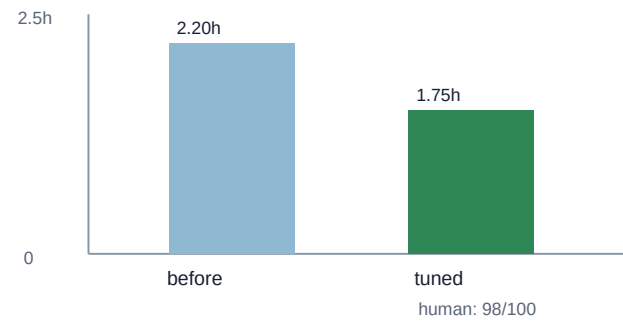
Client	Baseline	Tuned / corrected run	Measured outcome	Human score
Pi	214,639 input + 155,999 output; 102 turns. Wall time not persisted.	143,422 + 99,289; 63 turns; ~1h40m service window.	Both static/runtime clean. Tuned run used 34.5% fewer total tokens and 38.2% fewer turns.	85 / 100
Pi-Wiggum	95,341 + 40,455; 42 turns; 7,924.398 s.	161,872 + 109,356; 60 turns; 6,317.143 s.	Both passed 7/7 logic tests. Tuned run was 20.3% faster despite roughly twice the tokens.	98 / 100
OpenCode	First 32K-context attempt was stopped and archived; not a valid baseline.	715,325 + 253,739; 121 turns; 132 tool calls; ~3h38m42s; 131K context.	Static/runtime clean; independent elevator logic test 7/7. Complete working solution, but much more meandering.	80 / 100

Pi · total tokens



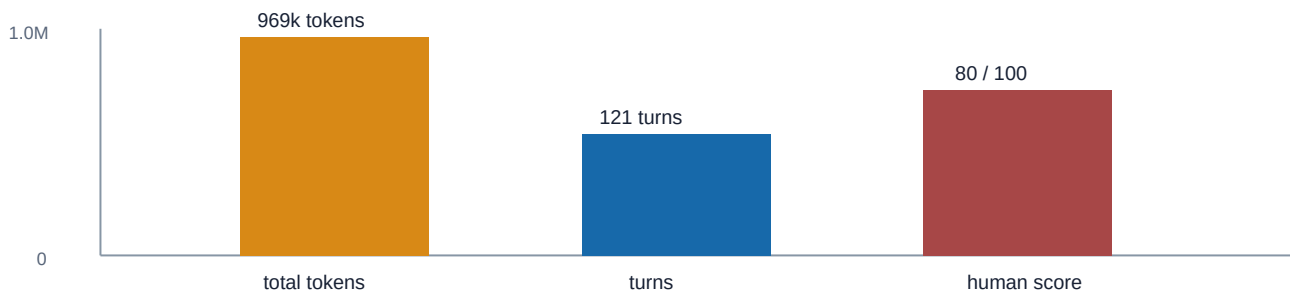
Tuned Pi needed 34.5% fewer total tokens and 38.2% fewer turns.

Pi-Wiggum · wall time



The tuned run saved 26m47s, a 20.3% wall-time improvement.

OpenCode · corrected full-context run



No valid before bar is shown: the first attempt advertised only 32K context and was stopped before producing a result.

Best operating procedure

1. Keep `llama-swap.service` as the single model switchboard and expose the OpenAI-compatible endpoint at `http://127.0.0.1:8080/v1`.
2. For Pi or OpenCode agentic coding, select `qwen3.8-27b-think`. Keep the client context limit aligned with the server: 131,072 for this profile. OpenCode evaluation required an explicit `--lms-context-length 131072` override.
3. Keep `--ctk f16 --ctv f16`, flash attention, `--parallel 1`, MTP draft length 1, reasoning budget 8192, and presence penalty 0.
4. Use request-level `reasoning_effort=low` for routine coding and `medium/xhigh` only when the task justifies longer reasoning. Avoid `none` as a shortcut.
5. Use `qwen3.8-27b-mtp` for a real 256K-context requirement, accepting the q8-KV deep-context penalty. Use the non-MTP alias when speculative behavior is undesirable.
6. Keep the Unsloth Qwen3.6 UD quant for balanced general-purpose work, and use smaller Gemma for quick transformations. Agents-A1 remains the high-throughput long-horizon alternative; Muse Glimmer remains useful for multimodal ATEM tool work.
7. After changing a model alias or context, run `~/Code/intel/bin/llm-swap configure` so Pi and OpenCode receive the same model list, modalities, context, and output limits.

```
systemctl --user restart llama-swap.service
~/Code/intel/bin/llm-swap configure
```

```
opencode -m local-b70/qwen3.8-27b-think
pi --provider local-b70 --model qwen3.8-27b-think
```

What to keep, what to avoid

Keep

- Current llama.cpp refresh and IntelLLVM/SYCL build.
- Level Zero, oneDNN, FP16 kernels, host-memory fallback, flash attention.
- JIT device compilation with `GGML_SYCL_DEVICE_ARCH` unset.
- One-token MTP for Qwen3.8; f16 KV for deep-context agents.
- Prompt caching with one serialized agent slot.

Avoid or treat as experimental

- Qwen3.8 q8 KV for deep 128K reasoning when latency matters.
- ngram-mod on this SYCL backend.
- Nemotron MTP as a default; it regressed and could stall on longer work.
- AOT BMG architecture flags that remove the >4 GB buffer path.
- Removing the DPC++ compiler/shared development packages if rebuilds are expected. Eclipse plugins and some CCL/debugger extras are not runtime requirements, but the compiler/toolchain is.

Limitations and interpretation

- The before/after runs compare the older qwen3.8-27b profile with the tuned qwen3.8-27b-think profile. They are practical end-to-end comparisons, not single-variable laboratory controls.
- The Pi baseline's exact wall time was not persisted, and the long-run harness did not retain separate prompt-processing and generation rates. Short controlled llama-server measurements remain the authoritative throughput data.
- The automated browser probes establish load, animation, canvas, and error-free execution; they do not fully judge visual or behavioral quality. The human scores supplied for this report are therefore valuable complementary evidence.
- The first OpenCode run was invalid for apples-to-apples comparison because it advertised 32K context. It was stopped, checkpointed, and excluded from the final performance table.

Conclusion: profiling changed the recommendation from “run the largest model that fits” to “match cache precision and context to the workload.” On this hardware, that distinction is worth the difference between a model that feels sluggish at depth and one that can complete a long agentic coding task reliably.

Prepared from the local Intel Arc Pro B70 measurements, llama-swap configuration, llama.cpp build notes, and archived Pi/Pi-Wiggum/OpenCode evaluations. Human client scores: Pi 85, Pi-Wiggum 98, OpenCode 80.